

Lecture 01: First Program and Variables

Introduction to Programming with Kotlin

Agenda

1. Your first Kotlin program
2. Reading from the console
3. Variables with `val`
4. Special characters in strings
5. String templates

What is Kotlin?

Kotlin is a modern, concise programming language that runs on the JVM (and elsewhere).

- Designed for clarity and safety (null-safety, sensible defaults)
- Fully interoperable with Java
- Used for Android, backends, multiplatform apps, and teaching
- Official language for Android; popular in industry and education

Hello, Kotlin

The shortest runnable program:

```
1 fun main() {  
2     println("Hello, Kotlin!")  
3 }
```

- `fun main()` — entry point (no class required for scripts/simple apps)
- `println` prints a line and moves to the next line
- Save as `Hello.kt`, then compile and run (IDE or `kotlinc`)

How a Kotlin program runs

Rough pipeline

1. You write source in a `.kt` file
2. The Kotlin compiler produces JVM bytecode (or another target)
3. The runtime executes that bytecode

In this course we focus on writing clear programs, not on build tools. Use IntelliJ IDEA / Android Studio / VS Code with a Kotlin plugin if you like.

print VS println

```
1 fun main() {  
2     print("Hello")  
3     print(" ")  
4     println("Kotlin")  
5     println("Next line")  
6 }
```

Output:

- Hello Kotlin (same line), then
- Next line

Try this

Change the messages and run again. What happens if you use only `print`?

Quiz 1 — First program

Quiz

What is the entry point of a simple Kotlin application?

1. `public static void main(String[] args)`
2. `fun main()`
3. `class Main`
4. `System.out.println`

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

2. `fun main()`

Kotlin uses `fun main()` as the program entry point. You may also write `fun main(args: Array<String>)`, but for beginners the empty-parameter form is enough.

Reading from the console

Programs become useful when they react to input.

- `readln()` — reads one line of text from standard input
- Returns a `String` (text)
- Prefer `readln()` in modern Kotlin (1.6+); older code may use `readLine()`

Common mistake

`readLine()` returns `String?` (nullable). `readln()` returns a non-null `String` and throws if input ends.

Greeting by name

```
1 fun main() {  
2     print("Enter your name: ")  
3     val name = readln()  
4     println("Hello, $name!")  
5 }
```

- Prompt with print (stay on the same line)
- Store the result in a variable
- Use it later in println

Variables: `val`

A **variable** is a named place that holds a value.

- `val` — read-only binding (preferred by default)
- You give a name, optionally a type, and a value
- Names: letters, digits, underscore; start with a letter
- Style: camelCase (`userName`, `age`)

Mental model

`val name = readln()` means: “name refers to that string, and we will not reassign name to something else.”

Declaring `val`

```
1 fun main() {  
2     val greeting = "Welcome"  
3     val year = 2026  
4     val pi = 3.14  
  
5  
6     println(greeting)  
7     println(year)  
8     println(pi)  
9 }
```

Kotlin **infers** types when the value is clear: `String`, `Int`, `Double`. You can write types explicitly: `val year: Int = 2026`.

Reading numbers (preview)

Input is always text. Convert when you need a number:

```
1 fun main() {  
2     print("Enter age: ")  
3     val age = readln().toInt()  
4     println("Next year you turn ${age + 1}")  
5 }
```

- `toInt()` parses an integer string
- Invalid input (e.g. "abc") causes an exception — we handle that later

Try this

Ask for two numbers, convert both with `toInt()`, and print their sum.

Quiz 2 — Variables and input

Quiz

Which declaration is correct and idiomatic for a name that will not be reassigned?

1. `var name = readln()`
2. `val name = readln()`
3. `String name = readln()`
4. `name := readln()`

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer — Quiz 2

Answer

2. `val name = readln()`

Prefer `val` when the binding does not change. `var` is for mutable bindings (next lectures).
Option 3 is Java-style; option 4 is not Kotlin.

Special characters in strings

Some characters are hard to type “as themselves” inside quotes. Kotlin uses **escape**

sequences with a backslash:	Escape	Meaning
	<code>\n</code>	new line
	<code>\t</code>	tab
	<code>\\</code>	a literal backslash
	<code>\"</code>	a double quote inside a string
	<code>\\$</code>	a literal dollar sign

Escapes in action

```
1 fun main() {  
2     println("Line 1\nLine 2")  
3     println("Name:\tAda")  
4     println("She said: \"Hello\"")  
5     println("Path: C:\\Users\\Student")  
6 }
```

Raw strings (triple quotes) can hold multiline text with fewer escapes:

```
1 val poem = """  
2 First line  
3 Second line  
4 """.trimIndent()
```

String templates

Build text from variables without awkward concatenation.

- `$name` — insert a simple variable
- `${expression}` — insert any expression
- Prefer templates over `+` for readability

Templates vs concatenation

```
1 fun main() {  
2     val a = 3  
3     val b = 5  
4     // Template (preferred)  
5     println("$a + $b = ${a + b}")  
6     // Concatenation (works, noisier)  
7     println("" + a + " + " + b + " = " + (a + b))  
8 }
```

Output: 3 + 5 = 8

Common mistake

Writing `$a + b` only substitutes `a`. Use `${a + b}` for the sum.

Putting it together

```
1 fun main() {  
2     print("First name: ")  
3     val first = readln()  
4     print("Year of birth: ")  
5     val birthYear = readln().toInt()  
6     val age = 2026 - birthYear  
7     println("Hello, $first!\nYou turn $age this year.")  
8 }
```

Try this

Add a last name and print Full name: First Last using a template.

Quiz 3 — Templates

Quiz

What does this print? Assume `val n = 4`.

```
println("n squared is ${n * n}")
```

1. `n squared is ${n * n}`
2. `n squared is 4 * 4`
3. `n squared is 16`
4. A compile error

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

3. `n squared is 16`

`${n * n}` evaluates the expression and inserts the result into the string.

Summary

- `fun main()` starts the program; `println` writes output
- `readln()` reads a line of text from the console
- Prefer `val` for bindings that do not change
- Escape special characters with `\n`, `\t`, ...
- Use string templates: `$x` and `${...}`

Looking ahead

Next lecture: the `Int` type, integer division, mutable variables (`var`), floating-point numbers, and other integer sizes (`Long`, `Short`, `Byte`).